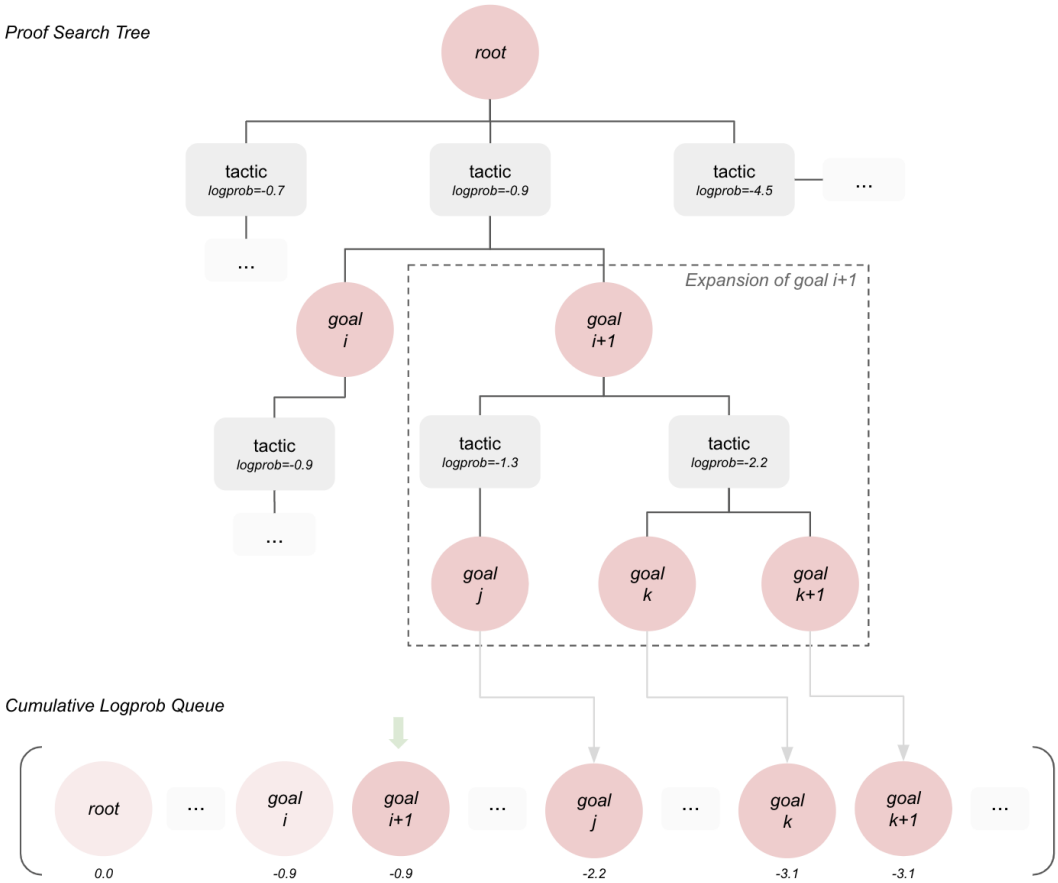

Figure 1: Proof search consists in maintaining a proof tree where multiple tactics are explored for each goal, starting from the root goal. Goals are expanded by cumulative (tactic) logprob priority.

The hyperparameters we chose to evaluate our models attempt to minimize the variance in the evaluation process while using the least amount of compute. Decreasing the number of expansions per goal e increases the variance as we less systematically explore the action space when expanding each goal. The e value can be taken quite high as each auto-regressive proof step generation uses the same query for a given goal and can therefore be batched together. Increasing e too much may also hurt performance given the breadth first nature of the cumulative logprob ordering. Increasing the number of attempts per proposition a decreases variance and consistently improves performance up to reasonably high values (We use $a = 32$ attempts for our final benchmarking). We found that $a = 4$ limited the variance in our evaluations while remaining tractable given the amount of compute we had available. Finally the proof search depth d has little impact on variance but naturally improves performance (we take $d = 128$ to evaluate our models and $d = 256$ for our final benchmarking).

The number of expansions we use per proof search may appear as relatively low, but it's important to realize that it already requires a substantial amount of compute as each expansion consists in the auto-regressive generation of $e = 32$ tactics (generally hundreds of tokens and therefore forward passes each). Empirically, the hyperparameters we chose, require on average around $\sim 1k$ GPU.hours (with V100s) to evaluate our 700m parameters model (which leverages GPT-3's sparse attention as well as key-value caching).

4.5 Pre-training

We study the effect of pre-training on the performance of our models. We pre-train our models on both GPT-3's post-processed version of CommonCrawl as well as a more reasoning-focused mix of Github, arXiv and Math StackExchange.

Github is downloaded using BigQuery[4] and filtered to only include deduplicated files from selected programming languages (excluding markdown, stylesheets, HTML). arXiv is downloaded using Bulk Data Access[5] and filtered to only include articles labeled as Mathematics and whose LaTeX source is available. Math StackExchange is downloaded from their snapshot on the Internet Archive[6] and post-processed to remove HTML tags and correlate questions and answers. We demote the mix reported in the table below as *WebMath*:

Table 1: Mix and source of data involved in the *WebMath* dataset.

Dataset	Size	Mix
Github	23 GB	33%
arXiv Math	10 GB	33%
Math StackExchange	2 GB	33%

4.6 Synthetic Datasets

Despite being among the largest formal mathematics libraries, the Metamath library remains scarce in the context of deep learning, especially in light of the advantages demonstrated on various NLP tasks by pre-training on large corpora. Also *set.mm* mostly focuses on well-known high-level theorems and does not include a large number of technical lemmas resembling the type of mathematics exercises used as curriculum for humans. Finally, Metamath lacking high level tactics such as HOL Light's `ARITH_RULE`[7], or Lean's `ring`[8], it is critical to ensure that our models are capable of proving at least basic technical theorems generally handled by high-level tactics in other systems (in domains such as arithmetic or ring equalities and inequalities)

To achieve this goal we designed synthetic datasets allowing us to generate proofs for each of these domains at will while controlling precisely by how many proofs we augment our training set.

We describe below the synthetic datasets we designed and report in section 5 the sample complexity associated with these synthetic tasks.

4.6.1 *n-digit* Arithmetic

We synthetically generate proofs for arithmetic formulas such as $11 * 22 = 242$ by following the basic algorithm for addition and multiplication, repeatedly applying theorems such as `decadd`[9] or `decaddc`[10]. Divisions and subtractions are translated to their equivalent additions and multiplications theorems in one proof step. We also support generating modulos and exponentiations.

We accept one hyperparameter for these synthetic proof generators, $ndigits$ which controls the number of digits involved in these arithmetic tasks. When generating a new proof we sample uniformly in $[-10^{ndigits}, 10^{ndigits}]$ each of the numbers involved. To illustrate the level at which Metamath operates, Table 2 shows the average number of proof steps generated as a function of $ndigits$ for each generator. These statements are generally proved with one tactic application in other higher-level systems, which is a good example of one of Metamath's drawbacks we identified earlier.

Table 2: Average number of proofsteps produced by our synthetic generators for $ndigits = 3, 9, 18$.

	3	9	18
Addition (in $\mathbb{Z}$)	19	48	94
Division	13	93	292
Modulo	25	82	206
Exponentiation	7	27	68

[4] `https://console.cloud.google.com/marketplace/details/github/github-repos`

[5] `https://arxiv.com/help/bulk_data`

[6] `https://archive.org/details/stackexchange`

[7] `https://www.cl.cam.ac.uk/~jrh13/hol-light/HTML/ARITH_RULE.html`

[8] `https://leanprover-community.github.io/mathlib_docs/algebra/ring/basic.html#ring`

[9] *Metamath Proof Explorer - decadd* `http://us.metamath.org/mpeuni/decadd.html`

[10] *Metamath Proof Explorer - decaddc* `http://us.metamath.org/mpeuni/decaddc.html`

Our goal is to leverage these synthetic generators to ensure our models are confident when faced with such subgoals in order to mitigate the large number of proof steps they require.

4.6.2 Ring Algebra

Our ring equalities generator is largely inspired by the INT inequality generator [45] . They propose an inequality generator that starts from simple formulas (such as $A = A$) and iteratively transforms them into more complex equalities or inequalities using a predefined list of axioms (such as commutativity of addition or distributivity of addition-multiplication). At each transformation, the axiom to be applied is chosen uniformly.

Our generator operates similarly within the Metamath formalism based on theorems equivalent to the axioms they propose. We accept two hyperparameters, the number of variables $nbvar$ involved in the seed formulas (of the form $A = A$) as well as the number of theorems applied to transform the expression, denoted as $depth$. In addition, we use hand-crafted weights as we sample theorems in order to obtain formulas that we judged qualitatively better.

Here is a list of the theorems we use and their associated sampling weights.

Table 3: Metamath theorems use by our Ring Algebra synthetic generators. Theorems are available in the Matmath Proof Explorer.

Theorem	Weight	Description
`eqcomd`	1	Commutative law for class equality.
`int-addcomd`	1	Addition commutativity.
`int-addassocd`	1	Addition associativity.
`int-mulcomd`	1	Multiplication commutativity.
`int-mulassocd`	1	Multiplication associativity.
`int-leftdistd`	3	Left distribution of multiplication over addition.
`int-rightdistd`	3	Right distribution of multiplication over addition.
`int-sqdefd`	5	Definition of the square.
`muladdd2`	5	Product of two sums

Examples of equalities produced by the generator:

$$
\begin{aligned}
ABBA(AB)^2 + (C + A) &= A + (ABBA)^2 + C \\
(AA)^2 &= A^2AA \\
((BA + CA)^2)^2 &= (BA + CA)^2(BAAB + ACCA + BAAC + ABCA) \\
((A + B)^2)^2(A + A) &= ((A + B)^2(AB + AB + AA + BB) + (A + B)^2(AB + AB + AA + BB))A
\end{aligned}
$$

4.6.3 Default *augmented* Dataset

By default in all of our experiments we add synthetically generated proofs to the dataset extracted from *set.mm* as shown in Table 4. We'll denote this dataset as our *augmented* dataset. The synthetically generated proofs account for approximately 1% of our training data which empirically appeared as big enough to achieve decent performance on the tasks we cared about and small enough not to hurt performance on the *valid* set, especially for small models. We attempted scaling the portion of synthetic proofs to 5% of the dataset and found out that it hurt performance for the model sizes we studied. It is nonetheless possible that including more synthetic data may turn out to be beneficial for larger models than the ones studied in this paper.

4.7 Learned Value Function

To achieve better performance, we also iteratively train a value function to guide the proof search, in place of the cumulative logprob priority described above.

We implement the value function by means of an *outcome objective* as follows. Any time we attempt to prove a statement, we will generate a significant number of intermediate goals. Some of these